
Sentinelle Numérique

Plateforme de Détection de Fake News

Livrable 1

Dossier de Conception

Groupe 2 — Spécialiste Deepfake Vidéo

Méthode SCRUM · Axe 1 : Analyse Visuelle et Multimédia

MEMBRES DU GROUPE

ABDOUL-HAMID OUMAROU

ATEH FRANK ATEH

BOUGNA BELVINE MAILLEVA

KOS NDOLIKE DANIEL NATHAN

NKWAWYA NJIONTCHOU JOEL CONSTATIN

Année Académique 2025–2026

Table des matières

1	Présentation du Projet	3
1.1	Contexte général	3
1.2	Problématique	3
1.3	Solution proposée	3
1.4	Objectif technique	4
1.5	Méthode de développement : SCRUM	4
2	Diagramme de Cas d'Utilisation	5
2.1	Objectif du diagramme	5
2.2	Le diagramme	5
2.3	Explication détaillée de chaque élément	5
2.3.1	La frontière du système (rectangle en pointillés)	5
2.3.2	Les acteurs	5
2.3.3	Les cas d'utilisation (ellipses)	6
2.3.4	Les relations	7
3	Diagramme de Séquence	8
3.1	Objectif du diagramme	8
3.2	Le diagramme	8
3.3	Les participants et leurs lignes de vie	8
3.4	Les messages — détail de chaque étape	8
3.5	Convention de lecture	10
4	Diagramme de Classes	11
4.1	Objectif du diagramme	11
4.2	Le diagramme	11
4.3	Explication de chaque classe	12
4.3.1	Classe Vidéo	12
4.3.2	Classe ServiceDeepfake	12
4.3.3	Classe ResultatAnalyse	12
4.3.4	Classe AnalyseurClignotements	12
4.3.5	Classe CalculateurScore	12
4.3.6	Classe AnalyseurLevres	13
4.3.7	Relations entre classes	13

5	Schéma d'Architecture des Microservices	14
5.1	Objectif du schéma	14
5.2	Pourquoi une architecture microservices ?	14
5.3	Le schéma	14
5.4	Explication de chaque couche	14
5.4.1	Couche 1 — Interface Utilisateur (Tableau de Bord)	14
5.4.2	Couche 2 — Passerelle API	15
5.4.3	Couche 3 — Microservice Deepfake Vidéo (votre moteur)	15
5.4.4	Couche 4 — Analyse IA (APIs Externes)	16
5.4.5	Couche 5 — Stockage	16
6	Planification SCRUM — Product Backlog	17
6.1	Organisation des sprints	17
6.2	Product Backlog complet	17
	Conclusion	19

Chapitre 1

Présentation du Projet

1.1 Contexte général

Dans un monde saturé de contenus synthétiques, les vidéos truquées — communément appelées *deepfakes* — représentent une menace croissante pour la confiance numérique. Grâce aux progrès fulgurants de l'intelligence artificielle générative, il est désormais possible de produire des vidéos réalistes dans lesquelles le visage ou la voix d'une personne est entièrement simulé par un algorithme.

Les entreprises, les médias et le grand public ont un besoin urgent d'outils capables de **certifier l'authenticité d'une vidéo**. Le défi ne se limite plus à distinguer le vrai du faux : il s'agit d'apporter des preuves techniques mesurables de manipulation.

1.2 Problématique

Comment détecter automatiquement si une vidéo courte (de type TikTok/Reels) a été générée ou manipulée par une intelligence artificielle, en s'appuyant sur des indicateurs physiques mesurables et quantifiables ?

1.3 Solution proposée

Notre groupe développe un **microservice d'analyse vidéo deepfake** qui s'intègre dans la plateforme *Sentinelle Numérique*. Ce microservice détecte deux anomalies physiques caractéristiques des visages synthétiques :

- **Les micro-clignements d'yeux anormaux** : un visage généré par IA ne cligne pas naturellement. Les clignements sont absents, trop réguliers ou mécaniques. Nous mesurons la fréquence et la durée des clignements via l'API **MediaPipe** (Google), qui détecte les *landmarks* faciaux image par image.
- **Les artefacts de synchronisation labiale** : dans un deepfake, le mouvement des lèvres n'est jamais parfaitement aligné avec l'audio. Nous mesurons cet écart

via **SyncNet** en comparant le flux visuel de la bouche et le flux audio.

Le système retourne un **score de confiance** indiquant la probabilité que la vidéo soit authentique, par exemple :

Score de confiance : 42% (Faible)

1.4 Objectif technique

Développer une application distribuée en **architecture microservices**, exposée via une API REST, intégrant des APIs d'IA spécialisées, visualisable sur un tableau de bord web, et déployable via Docker.

1.5 Méthode de développement : SCRUM

Notre équipe de **5 membres** applique la méthode **SCRUM** avec 4 sprints d'une semaine chacun. Les rôles sont distribués comme suit :

Rôle SCRUM	Membre(s)	Responsabilité
Scrum Master	Ateh	Organisation des mêlées, gestion GitHub, suppression des obstacles
Product Owner	Bougna	Rédaction du backlog, priorisation, liaison avec l'enseignant
API Gateway & Architecture	Abdoul, Bougna	Passerelle API, routage, architecture microservices
Moteur de détection yeux	Ateh, Nkwawya	Détection des clignements via MediaPipe
Synchro labiale & Tableau de bord	Kos, Abdoul, Nkwawya	Synchronisation labiale via SyncNet + interface tableau de bord

Chapitre 2

Diagramme de Cas d'Utilisation

2.1 Objectif du diagramme

Le diagramme de cas d'utilisation est le premier diagramme comportemental de notre modélisation UML. Il répond à une question fondamentale : **qui interagit avec notre système, et pour faire quoi ?** Il ne s'intéresse pas au *comment* — c'est le rôle du diagramme de séquence — mais au *quoi* : quelles fonctionnalités le système offre-t-il et à quels acteurs ?

Ce diagramme permet à l'enseignant, au client, ou à tout membre de l'équipe de comprendre en un coup d'œil le périmètre fonctionnel du système, sans nécessiter de connaissances techniques.

2.2 Le diagramme

2.3 Explication détaillée de chaque élément

2.3.1 La frontière du système (rectangle en pointillés)

Le grand rectangle en tirets délimite **les limites exactes de notre logiciel**. Tout ce qui se trouve à l'intérieur de ce rectangle est développé par notre groupe. Tout ce qui est à l'extérieur représente le monde extérieur avec lequel notre système interagit.

Cette frontière, appelée *System Boundary* en UML, est essentielle : elle définit précisément le périmètre de notre responsabilité technique et évite toute confusion entre ce que notre code fait et ce que font les systèmes tiers.

2.3.2 Les acteurs

Un acteur représente **toute entité extérieure au système qui interagit avec lui**. Les acteurs ne sont jamais à l'intérieur de la frontière ; ils se trouvent toujours dehors.

- **Utilisateur (gauche)** — C'est la personne physique devant l'interface (un journaliste, un fact-checker, un internaute) qui soumet une vidéo suspecte et

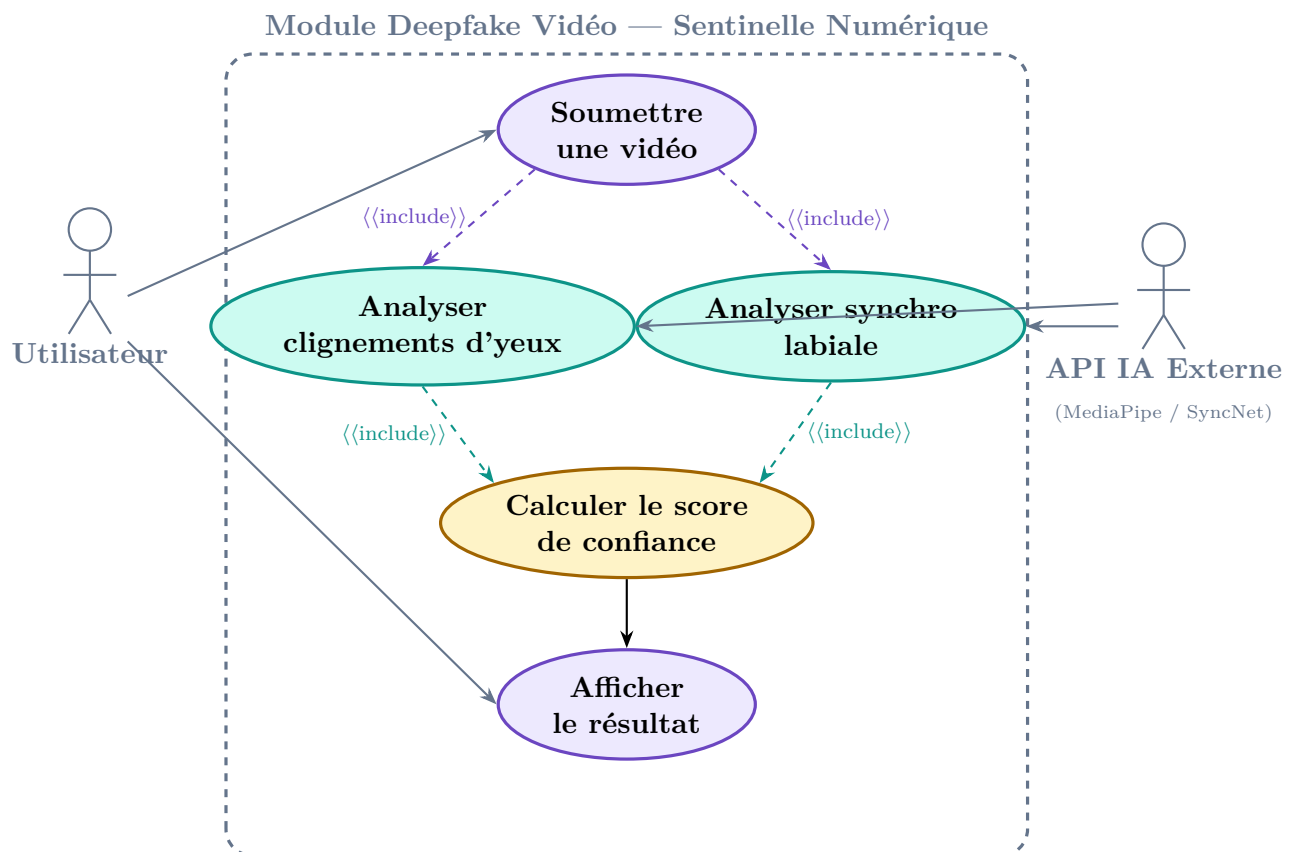


FIGURE 2.1 — Diagramme de cas d'utilisation — Module Deepfake Vidéo

attend un verdict. Il ne connaît pas les détails techniques de l'analyse ; il voit uniquement l'interface et le résultat. Il initie le processus (soumission) et en reçoit la conclusion (affichage du score).

- **API IA Externe (droite)** — Cet acteur représente les services d'intelligence artificielle que notre système appelle : **MediaPipe** pour la détection des clignements d'yeux, et **SyncNet** pour l'analyse de synchronisation labiale. Ils sont des acteurs car ils interagissent avec notre système depuis l'extérieur, mais nous ne les développons pas nous-mêmes.

2.3.3 Les cas d'utilisation (ellipses)

Chaque ellipse représente **une fonctionnalité observable que le système offre à ses acteurs**. Chaque couleur reflète une nature différente.

- **Soumettre une vidéo (violet)** — Point d'entrée de tout le flux. L'utilisateur fournit une URL ou un fichier vidéo. Le système valide le format, télécharge temporairement la vidéo, et déclenche automatiquement toute la chaîne d'analyse. Sans cette action, rien ne s'amorce dans le système.

- **Analyser clignements d’yeux** (*vert*) — Le système extrait les frames de la vidéo, détecte le visage, et mesure la fréquence et la durée des clignements. Un humain cligne naturellement toutes les 3 à 8 secondes. Un deepfake présente des clignements absents ou mécaniquement réguliers. Ce cas produit un *sous-score*.
- **Analyser synchro labiale** (*vert*) — Le système compare le mouvement des lèvres (flux vidéo) et la parole (flux audio). Dans un deepfake, il existe toujours un écart mesurable entre les deux. SyncNet quantifie cet écart et produit un second *sous-score*.
- **Calculer le score de confiance** (*ambre*) — Le système combine les deux sous-scores via une formule de pondération :

$$\text{Score final} = (\text{score_yeux} \times 0.5) + (\text{score_lèvres} \times 0.5)$$

Ce cas est en ambre car il est d’une nature différente ; c’est une **décision**, non une analyse brute.

- **Afficher le résultat** (*violet*) — Le tableau de bord présente le score, le niveau de confiance (Faible / Moyen / Élevé), et une explication visuelle à l’utilisateur. Il est en violet — comme “Soumettre une vidéo” — car ces deux cas concernent directement l’**interface utilisateur**.

2.3.4 Les relations

- **Associations** (flèches pleines) — Elles signifient qu’un acteur *participe* à un cas d’utilisation. L’Utilisateur participe à “Soumettre” et “Afficher”. L’API IA participe aux deux analyses.
- **Relations** $\langle\langle\text{include}\rangle\rangle$ (flèches en tirets) — Elles signifient qu’un cas d’utilisation en déclenche *automatiquement et systématiquement* un autre. “Soumettre une vidéo” déclenche obligatoirement les deux analyses. Chaque analyse contribue obligatoirement au calcul du score. L’ordre est imposé par la logique métier, non par le choix de l’utilisateur. Ceci est différent de $\langle\langle\text{extend}\rangle\rangle$ qui serait conditionnel.

Chapitre 3

Diagramme de Séquence

3.1 Objectif du diagramme

Le diagramme de séquence modélise les **interactions dynamiques entre les composants du système dans l'ordre chronologique**. Là où le diagramme de cas d'utilisation répond à “qui fait quoi”, le diagramme de séquence répond à : **dans quel ordre exact et comment se parlent les composants ?**

C'est le diagramme le plus important pour la compréhension du flux technique de notre microservice. Il permet à un développeur de savoir précisément quelles méthodes implémenter et dans quel ordre les appeler.

3.2 Le diagramme

3.3 Les participants et leurs lignes de vie

Chaque rectangle en haut du diagramme représente un **participant** — un composant actif du système. La ligne pointillée verticale qui descend de chaque participant est sa **ligne de vie** : elle représente son existence au fil du temps.

Les **boîtes d'activation** (rectangles pleins sur les lignes de vie) indiquent les périodes où un composant est activement en train de traiter une requête. Dans notre diagramme, le Microservice est actif du message ③ au message ⑧, et les APIs IA sont actives uniquement pendant les appels d'analyse.

3.4 Les messages — détail de chaque étape

1. **soumettre(url_video)** — L'utilisateur colle une URL de vidéo suspecte dans le tableau de bord. C'est l'action déclencheuse de tout le flux. Sans elle, le système reste inactif.
2. **POST /analyser** — Le tableau de bord envoie une requête HTTP POST à l'API Gateway. Le corps de la requête contient l'URL en format JSON : `{"url":`

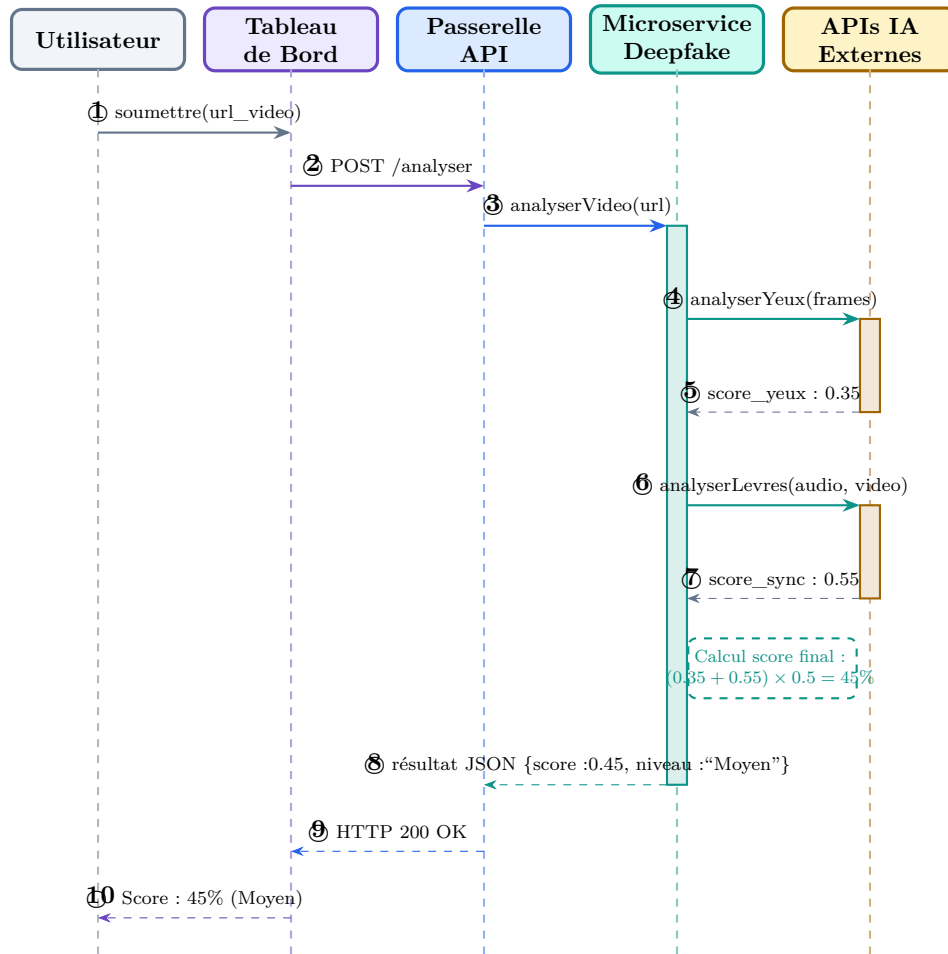


FIGURE 3.1 – Diagramme de séquence — Flux complet d'analyse d'une vidéo deepfake

"https://...", "type": "video"}.

3. **analyserVideo(url)** — L'API Gateway route la requête vers notre microservice deepfake. Ce routage est configuré dans les règles de la passerelle : toute requête sur `/deepfake/*` est dirigée vers ce microservice.
4. **analyserYeux(frames)** — Le microservice extrait les frames de la vidéo via `ffmpeg`, puis les envoie à MediaPipe pour la détection des *landmarks* faciaux et l'analyse des clignements.
5. **score_yeux : 0.35** — MediaPipe retourne un score normalisé entre 0 et 1. Un score de 0.35 signifie que peu de clignements normaux ont été détectés : comportement suspect.
6. **analyserLevres(audio, video)** — Le microservice extrait séparément l'audio et les frames de la région bouche, puis les envoie à SyncNet pour comparaison.

7. **score_sync : 0.55** — SyncNet retourne la distance cosinus entre le vecteur visuel et le vecteur audio. Un score de 0.55 indique une synchronisation partielle : certains artefacts détectés.
8. **résultat JSON** — Le microservice combine les scores, effectue le calcul interne $(0.35 + 0.55) \times 0.5 = 0.45$, et retourne un objet JSON structuré contenant le score, le niveau et l'horodatage.
9. **HTTP 200 OK** — L'API Gateway retransmet la réponse au tableau de bord avec le code de succès HTTP 200.
10. **Score : 45% (Moyen)** — Le tableau de bord affiche le verdict final à l'utilisateur avec une représentation visuelle (jauge colorée).

3.5 Convention de lecture

Élément	Signification
Flèche pleine ($\rightarrow$)	Appel direct ; le composant attend une réponse
Flèche pointillée ($- - \rightarrow$)	Retour / réponse à un appel
Rectangle sur la ligne de vie	Période d'activation (traitement actif)
Le temps s'écoule de haut en bas	Convention UML universelle

Chapitre 4

Diagramme de Classes

4.1 Objectif du diagramme

Le diagramme de classes modélise la **structure statique** du système : quelles sont les entités de notre code, quelles données possèdent-elles, quelles opérations peuvent-elles réaliser, et comment sont-elles reliées entre elles ? C'est le diagramme directement lié au code source que l'équipe va écrire : chaque classe deviendra un fichier `.py` dans notre projet Python.

4.2 Le diagramme

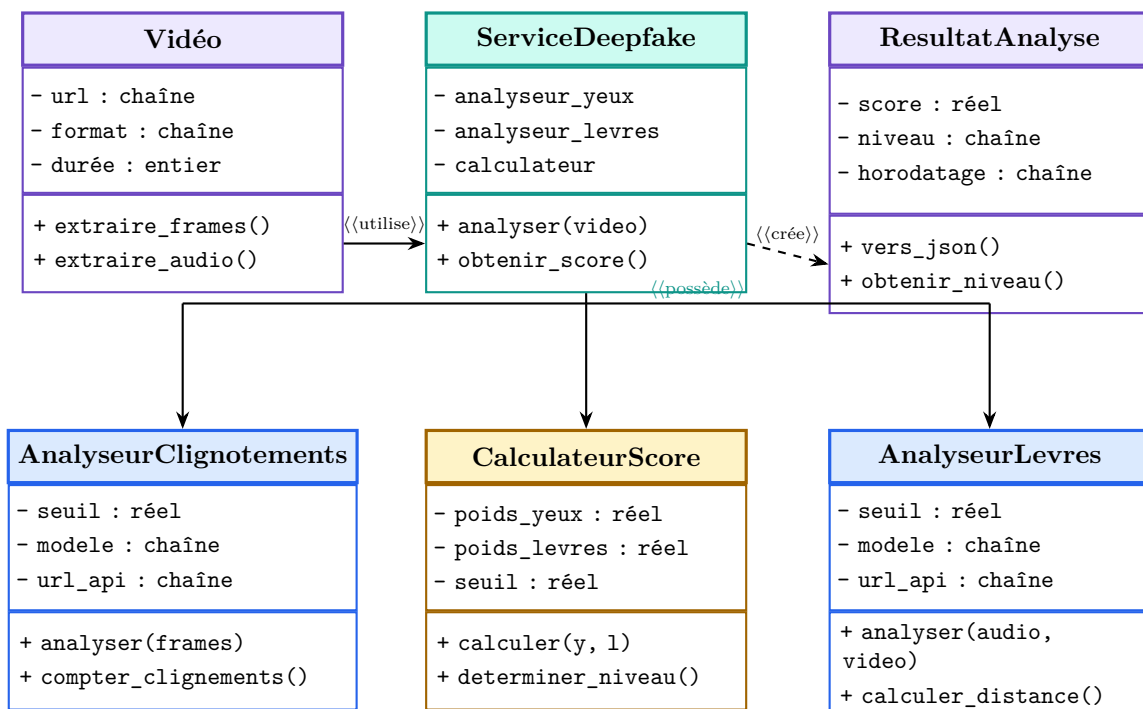


FIGURE 4.1 – Diagramme de classes — Module Deepfake Vidéo

4.3 Explication de chaque classe

4.3.1 Classe Vidéo

Cette classe représente la vidéo soumise par l'utilisateur. Elle encapsule les données brutes du média et offre deux méthodes essentielles pour préparer l'analyse :

- `extraire_frames()` découpe la vidéo en images individuelles via `ffmpeg`. Ces frames sont ensuite envoyées à l'analyseur des clignements.
- `extraire_audio()` extrait la piste sonore de la vidéo. Cet audio est envoyé à l'analyseur de synchronisation labiale.

4.3.2 Classe ServiceDeepfake

C'est l'**orchestrateur central** du système — le chef d'orchestre. Il coordonne les deux analyseurs, déclenche le calcul du score, et retourne un `ResultatAnalyse`. Il possède en attributs des références aux deux analyseurs et au calculateur, ce qui lui permet de les appeler dans le bon ordre.

La méthode `analyser(video)` est le point d'entrée principal du microservice : elle reçoit un objet `Vidéo`, déclenche les deux analyses en parallèle, et retourne l'objet résultat.

4.3.3 Classe ResultatAnalyse

Objet de sortie du système. Il contient trois attributs essentiels : `score` (réel entre 0 et 1), `niveau` (chaîne : "Faible", "Moyen" ou "Élevé"), et `horodatage` de l'analyse. La méthode `vers_json()` sérialise cet objet en JSON pour la réponse HTTP.

4.3.4 Classe AnalyseurClignotements

Elle encapsule l'appel à l'API MediaPipe. Elle reçoit les frames extraites, appelle l'API, compte les clignements, et normalise le résultat en un score entre 0 et 1. Le `seuil` définit la frontière entre un comportement normal et un comportement suspect : en dessous du seuil, la vidéo est considérée potentiellement synthétique.

4.3.5 Classe CalculateurScore

Elle implémente la formule de pondération finale :

$$S = (\text{score_yeux} \times \text{poids_yeux}) + (\text{score_lèvres} \times \text{poids_lèvres})$$

La méthode `determiner_niveau(score)` applique les seuils pour convertir le score numérique en étiquette lisible :

Score	Niveau	Interprétation
≥ 0.70	Élevé	Très probablement deepfake
0.40–0.69	Moyen	Suspicion modérée, vérification recommandée
< 0.40	Faible	Probablement authentique

4.3.6 Classe `AnalyseurLevres`

Elle encapsule l'appel à SyncNet. Elle reçoit les frames de la région bouche et le flux audio, appelle l'API, et retourne un score de synchronisation normalisé. La méthode `calculer_distance()` calcule la **distance cosinus** entre les vecteurs audio et vidéo : plus la distance est grande, plus l'artefact de synchronisation est important.

4.3.7 Relations entre classes

- `<<utilise>>` (`Vidéo` → `ServiceDeepfake`) — Le service reçoit un objet `Vidéo` en paramètre et utilise ses méthodes. C'est une *dépendance*.
- `<<crée>>` (`ServiceDeepfake` → `ResultatAnalyse`) — Le service instancie un objet `ResultatAnalyse` à la fin de son travail. C'est une relation de *création*.
- `<<possède>>` (`ServiceDeepfake` → `analyseurs` + `calculateur`) — Le service contient des références aux trois objets. C'est une *composition* : si le service est détruit, les analyseurs le sont aussi.

Chapitre 5

Schéma d'Architecture des Microservices

5.1 Objectif du schéma

Le schéma d'architecture présente la **vision technique globale** du système : comment les différents composants logiciels sont organisés, comment ils communiquent, et comment les données circulent de l'entrée (utilisateur) à la sortie (résultat). Il justifie également le choix d'une architecture microservices.

5.2 Pourquoi une architecture microservices ?

Dans une architecture *monolithique*, tous les modules seraient regroupés dans un seul programme. Si l'un plante, tout plante. Si l'on veut améliorer uniquement l'analyseur des lèvres, il faut redéployer tout le système.

L'architecture **microservices** résout ces problèmes : chaque service est indépendant, déployable séparément, et remplaçable. C'est particulièrement adapté à notre projet car les 8 groupes développent 8 moteurs d'analyse différents qui s'intègrent dans la même plateforme *Sentinelle Numérique*.

5.3 Le schéma

5.4 Explication de chaque couche

5.4.1 Couche 1 — Interface Utilisateur (Tableau de Bord)

Le Tableau de Bord est l'**unique point de contact** entre l'humain et le système. Développé en HTML/CSS/JavaScript (ou React), il permet à l'utilisateur de soumettre une URL de vidéo et d'afficher le résultat de manière visuelle : jauge de score, niveau de confiance coloré (vert / orange / rouge), et horodatage. Il communique avec la couche suivante uniquement via des appels HTTP REST.

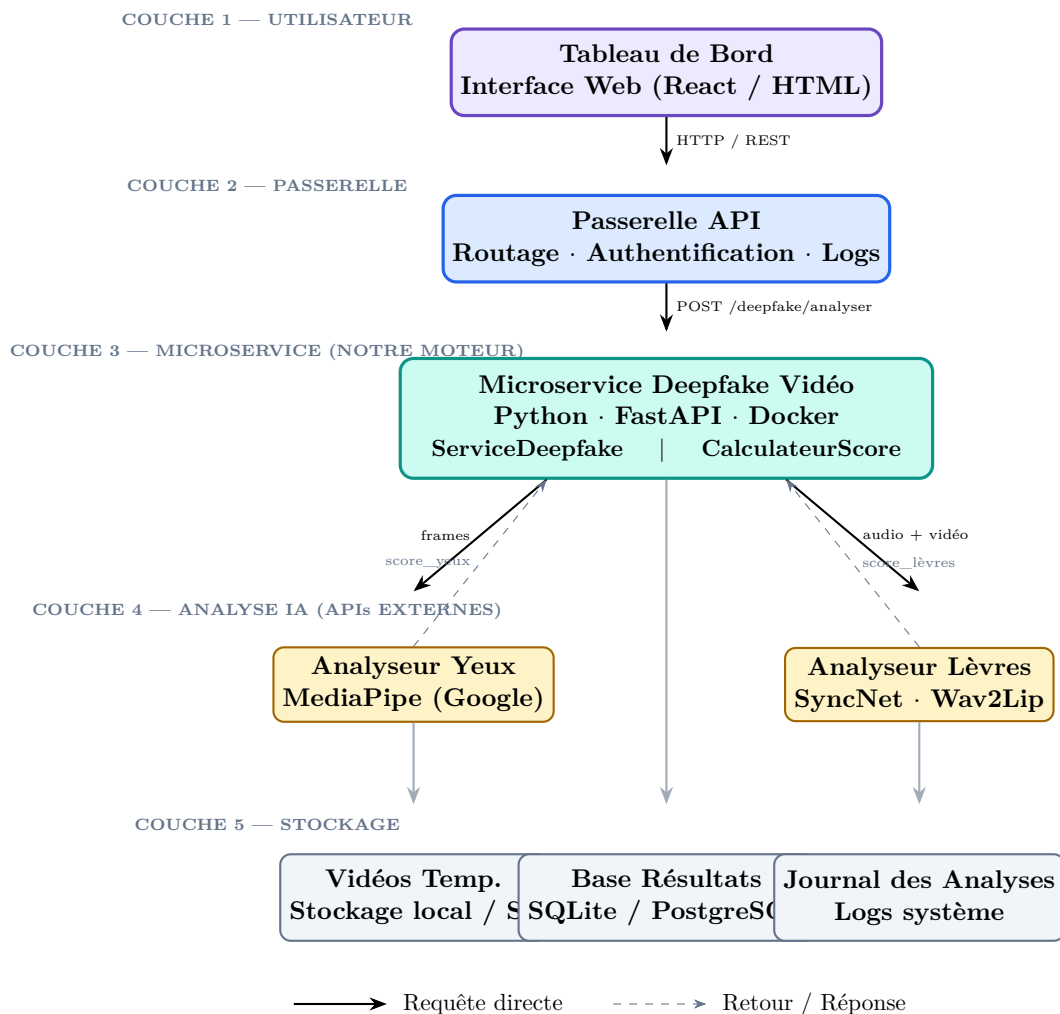


FIGURE 5.1 — Architecture en microservices — Module Deepfake Vidéo

5.4.2 Couche 2 — Passerelle API

La passerelle est le **point d'entrée unique** de toute l'infrastructure backend. Elle joue trois rôles essentiels :

- **Routage** : elle lit l'URL de la requête et la dirige vers le bon microservice. Les requêtes `/deepfake/*` vont vers notre microservice.
- **Authentification** : elle vérifie les tokens d'accès avant de transmettre la requête.
- **Journalisation** : elle enregistre chaque requête entrante pour le débogage et l'audit.

5.4.3 Couche 3 — Microservice Deepfake Vidéo (votre moteur)

C'est le cœur du travail de votre groupe. Construit avec **Python** et **FastAPI**, packagé dans un conteneur **Docker**, il reçoit la requête du Gateway, orchestre les deux analyses

(yeux + lèvres), calcule le score final, et retourne le résultat JSON. Il contient les classes `ServiceDeepfake` et `CalculeurScore` du chapitre précédent.

5.4.4 Couche 4 — Analyse IA (APIs Externes)

Deux sous-composants spécialisés appellent les APIs d'intelligence artificielle :

- **Analyseur Yeux** : appelle MediaPipe pour la détection des landmarks faciaux et le comptage des clignements.
- **Analyseur Lèvres** : appelle SyncNet ou Wav2Lip pour mesurer la distance cosinus entre le flux visuel (bouche) et le flux audio.

5.4.5 Couche 5 — Stockage

Trois éléments assurent la persistance des données :

- **Vidéos temporaires** : les fichiers vidéo sont stockés localement le temps de l'analyse, puis supprimés pour des raisons de confidentialité et d'espace disque.
- **Base de Résultats** : chaque analyse est archivée (score, horodatage, niveau) dans une base SQLite (développement) ou PostgreSQL (production).
- **Journal des analyses** : les logs techniques (erreurs, durées d'appels API, statistiques) sont enregistrés pour le débogage et l'amélioration continue.

Chapitre 6

Planification SCRUM — Product Backlog

6.1 Organisation des sprints

Notre équipe de 5 membres organise le travail en **4 sprints** d'une semaine chacun, conformément au planning de réalisation d'un mois défini dans le cahier des charges.

Sprint	Objectif principal	Livrable	Période
S1	Analyse, modélisation UML, conception	Livrable 1	Semaine 1
S2	Mise en place GitHub, API Gateway, AnalyseurYeux	—	Semaine 2
S3	AnalyseurLèvres, Dashboard, intégration	Livrable 2	Semaine 3
S4	Tests, corrections, préparation démonstration	Livrable 3	Semaine 4

6.2 Product Backlog complet

ID	User Story	Sprint	Critères d'acceptation	Priorité
US-01	En tant qu'utilisateur, je veux soumettre une URL de vidéo pour analyse	S2	URL validée, vidéo téléchargée, analyse lancée	Haute
US-02	En tant que système, je veux détecter les clignements d'yeux anormaux	S2	Score yeux normalisé [0–1] retourné en <2s	Haute
US-03	En tant que système, je veux analyser la synchronisation labiale	S3	Score sync normalisé [0–1] retourné en <3s	Haute

ID	User Story	Sprint	Critères d'acceptation	Priorité
US-04	En tant qu'utilisateur, je veux voir le score de confiance final	S3	Score en % avec niveau affiché sur le Dashboard	Haute
US-05	En tant que système, je veux une API Gateway opérationnelle	S2	Routage vers microservice fonctionnel, logs actifs	Haute
US-06	En tant qu'admin, je veux que les analyses soient journalisées	S3	Chaque analyse enregistrée en base avec horodatage	Moyenne
US-07	En tant que développeur, je veux containeriser le service avec Docker	S3	<code>docker-compose up</code> lance tout le système	Moyenne
US-08	En tant qu'équipe, je veux des tests unitaires pour les analyseurs	S4	Couverture de tests $\geq 70\%$ sur les classes IA	Moyenne
US-09	En tant qu'enseignant, je veux une démonstration en direct	S4	Score cohérent sur vidéo deepfake connue	Haute
US-10	En tant que visiteur GitHub, je veux un README clair	S4	Instructions d'installation et d'exécution complètes	Basse

Conclusion

Ce dossier de conception constitue le **fondement technique et méthodologique** du projet *Sentinelle Numérique — Groupe 2*. Les cinq artefacts produits — diagramme de cas d'utilisation, de séquence, de classes, schéma d'architecture, et product backlog SCRUM — forment un plan cohérent qui guidera le développement du microservice de détection de deepfakes vidéo.

La solution proposée repose sur deux piliers techniques **solides et mesurables** :

- La détection des micro-clignements d'yeux via **MediaPipe**, qui quantifie un phénomène physique caractéristique des visages synthétiques.
- L'analyse de synchronisation labiale via **SyncNet**, qui mesure mathématiquement l'écart entre flux audio et flux vidéo.

Ces deux approches produisent des **preuves techniques quantitatives** de manipulation, conformément aux exigences du cahier des charges. Le score final combiné offre une mesure objective, interprétable, et explicable à tout utilisateur non technique.

Le développement sera conduit selon la méthode **SCRUM** avec des commits progressifs et traçables sur GitHub, garantissant la visibilité de l'évolution du projet à toutes les parties prenantes, en particulier l'équipe enseignante.

Groupe 2 — Spécialiste Deepfake Vidéo
Sentinelle Numérique — Année Académique 2025–2026